DIFA-Projekte 2027

Gruppen C und D

Gruppe C

1. LangChain durch Literate Programming erlernen

Kontext

LangChain ist ein Open-Source-Framework für Python, das die Entwicklung von Applikationen mit grossen Sprachmodellen (LLMs) vereinfacht. Da verschiedene LLM-Anbieter unterschiedliche APIs und Antwortformate verwenden, standardisiert LangChain die Interaktion mit Sprachmodellen und ermöglicht so einen nahtlosen Anbieterwechsel.

Leitfragen

- Wie lassen sich zentrale LangChain-Konzepte (Chains, Prompts, Modell-Abstraktion) so aufbereiten, dass sie Schritt für Schritt nachvollziehbar sind?
- Wie kann das Prinzip des Literate Programming (Code und Erklärung untrennbar verwoben) methodisch konsequent umgesetzt werden?
- Welche typischen Fehler und Prompt-Iterationen sind lehrreich genug, um bewusst provoziert und dokumentiert zu werden?

Ziel

Erstellung von aufeinander aufbauenden Jupyter Notebooks, die LangChain nach dem Prinzip des Literate Programming Schritt für Schritt erklären: Jede Code-Zelle wird durch eine vorangehende Markdown-Zelle begründet («Was passiert hier und warum?»), Konzepte werden vor dem Code eingeführt.

Ergebnis

Eine Reihe didaktisch aufbereiteter, reproduzierbarer Jupyter Notebooks zu LangChain, die als Lernpfad für Dritte nutzbar sind und typische Stolpersteine sowie Prompt-Iterationen transparent dokumentieren.

Eignet sich als kompakter, didaktisch orientierter Einstieg in LLM-Frameworks und Literate Programming. Der Fokus liegt auf verständlicher Wissensvermittlung und sauberer Dokumentation von Code-Entscheidungen, nicht auf komplexer Anwendungsentwicklung. Damit eignet sich das Projekt gut für Studierende, die gerne strukturiert erklären und dokumentieren und dabei die Grundlagen von LLM-Frameworks erlernen möchten.

---

2. LangGraph durch Literate Programming erlernen

Kontext

LangGraph ist ein Open-Source-Framework für Python, mit dem LLM-Anwendungen als Graphen statt als lineare Abfolgen gebaut werden. Statt einer festen Kette von

Schritten modelliert man die Anwendung als Netzwerk aus Nodes (Verarbeitungsschritte wie «LLM aufrufen» oder «Tool ausführen») und Edges (Verbindungen, die den weiteren Ablauf festlegen). Ein gemeinsamer State wird zwischen den Schritten weitergereicht und von jedem Node aktualisiert.

Leitfragen

- Wie unterscheidet sich der graphbasierte Kontrollfluss von LangGraph von linearen Chain-Ansätzen wie in LangChain, insbesondere bei Schleifen und Verzweigungen?
- Wie lässt sich ein gemeinsamer State über mehrere kooperierende Agenten hinweg konsistent verwalten?
- Wie können Konzepte, Code und typische Fehler so verwoben werden, dass ein roter Faden über sechs aufeinander aufbauende Notebooks entsteht?

Ziel

Erstellung von sechs aufeinander aufbauenden Jupyter Notebooks, die LangGraph Schritt für Schritt nach dem Prinzip des Literate Programming erklären – inklusive bewusst provozierter Fehler, dokumentierter Prompt-Iterationen und reproduzierbarer Ausführung über CI/CD.

Ergebnis

Eine in sich geschlossene, reproduzierbare Notebook-Reihe zu LangGraph, die Nodes, Edges und State-Management praxisnah vermittelt und über CI/CD-Pipelines automatisiert lauffähig bleibt.

Eignet sich als vertiefender Einstieg in graphbasierte Multi-Agenten-Architekturen, die aktuell als Standard für «echte» Agenten-Systeme gelten. Der Fokus liegt auf dem Verständnis von Kontrollfluss, State-Management und Agenten-Kooperation. Das Projekt eignet sich für Studierende, die sich für Multi-Agenten-Architekturen interessieren und Konzepte gerne didaktisch aufbereiten.

Gruppe D

1. Integration der Tatoeba-API zur Erweiterung der Beispielsatz-Collection

Kontext

Eine bestehende, rudimentäre multilinguale Full-Stack-App verwaltet eine MongoDB-Datenbank mit Vokabeleinträgen in fünf Sprachen (Deutsch, Englisch, Französisch, Italienisch, Spanisch). Vielen dieser Einträge fehlen kontextgerechte Beispielsätze.

Leitfragen

- Wie lässt sich die öffentliche Tatoeba-REST-API zuverlässig und effizient in eine bestehende Datenpipeline integrieren (inkl. Fehlerbehandlung und Rate-Limiting)?

- Wie kann eine automatisierte Zuordnung von Beispielsätzen zur korrekten Wortbedeutung erfolgen, wenn ein Wort mehrdeutig ist (z. B. «Bank» als Möbelstück vs. Geldinstitut)?
- Wie werden die neu gewonnenen Daten sicher und konsistent in MongoDB geschrieben?

Ziel

Automatisierte Anreicherung der bestehenden Vokabeleinträge mit passenden, kontextgerechten Beispielsätzen über die Tatoeba-API, inklusive semantischer Disambiguierung mehrdeutiger Begriffe.

Ergebnis

Eine erweiterte MongoDB-Datenbank mit deutlich mehr korrekt zugeordneten Beispielsätzen sowie eine wiederverwendbare Anbindung an die Tatoeba-API mit robuster Fehlerbehandlung.

Eignet sich als praxisnaher Einstieg in die Arbeit mit externen REST-APIs. Der Fokus liegt auf sauberer API-Integration und semantischer Filterlogik. Das Projekt eignet sich gut für Studierende, die praktische Erfahrung mit Datenanreicherung und API-Anbindung an bestehende Full-Stack-Anwendungen sammeln möchten.

---

2. Vergleich von WebScraping-Technologien

Kontext

Ein bestehendes Vergleichsportal für Gymivorbereitungs-Anbieter nutzt drei Scraping-Methoden – Puppeteer, ScrapeGraphAI und Bright Data – um Kursdaten automatisiert von zwölf Anbieter-Websites zu extrahieren. Die technische Infrastruktur (Datenbank, Logging, Evaluierungsframework) ist bereits vollständig vorhanden und wiederverwendbar.

Leitfragen

- Wie lässt sich das LLM-basierte Scraping-Framework Crawl4AI in die bestehende Infrastruktur integrieren?
- Wie schneidet Crawl4AI im direkten, empirischen Vergleich zu den drei bestehenden Methoden hinsichtlich Genauigkeit, Robustheit und Aufwand ab?
- Welche Kriterien eignen sich für einen fairen, methodischen Vergleich über mehrere Scraping-Ansätze hinweg?

Ziel

Erweiterung des bestehenden Portals um Crawl4AI und empirischer Vergleich seiner Leistungsfähigkeit mit den drei bereits implementierten Scraping-Technologien anhand von fünf ausgewählten, bereits abgedeckten Anbietern.

Ergebnis

Eine vierte, LLM-basierte Scraping-Methode ist im Portal integriert; ein dokumentierter, empirisch belegter Methodenvergleich zeigt Stärken und Schwächen der vier Ansätze auf.

Eignet sich als fokussiertes Vergleichsprojekt im Bereich Web Scraping, bei dem die technische Grundinfrastruktur bereits steht und nicht neu erstellt werden muss. Der Fokus liegt auf der Integration eines neuen Frameworks und dem methodisch sauberen, empirischen Vergleich, nicht auf dem Aufbau der Infrastruktur selbst. Das Projekt eignet sich für Studierende, die sich für den praktischen und empirischen Vergleich verschiedener LLM- und klassischer Scraping-Ansätze interessieren.

---

3. Zod-Validierung und Zustand-State-Management im Gymi-Vergleichsportal

Kontext

In einer bestehenden Next.js Gymi-Vergleichsportal wurden Zod (TypeScript-Bibliothek zur Laufzeitvalidierung) und Zustand (leichtgewichtige State-Management-Bibliothek für React) als zentrale Architekturkomponenten eingesetzt. Zod validiert Daten an den Systemgrenzen (zwischen Datenbank, Scraping-Pipeline und Benutzeroberfläche), Zustand verwaltet den globalen State, insbesondere die Anbieterauswahl für den Direktvergleich und persistierte UI-Präferenzen. Beide Komponenten sind funktional implementiert, aber in der schriftlichen Arbeit bislang nicht mit vergleichbarer Tiefe dokumentiert wie z. B. die Scraping-Pipeline.

Leitfragen

- Welche Designentscheidungen liegen der Wahl von Zod und Zustand zugrunde, und
- Wie lässt sich eine belastbare automatisierte Testabdeckung für Zod-Schemas, MCDA-Scoring-Funktionen und Zustand-Stores aufbauen?
- Wie muss ein Dokumentationshandbuch gestaltet sein, damit neue Entwickler schnell in die Codebasis einsteigen können?

Ziel

Vollständige Dokumentation der Zod-Schemas und Zustand-Stores mit Architecture Decision Records sowie Aufbau einer automatisierten Testabdeckung (Unit Tests) für Zod-Schemas, MCDA-Scoring-Funktionen und Zustand-Stores.

Ergebnis

Vollständig dokumentierte und mit Architecture Decision Records versehene Zod- und Zustand-Komponenten, eine neu aufgebaute Jest-Testsuite für die entsprechenden Module sowie ein Entwicklerhandbuch für den Einstieg in die Codebasis.

Eignet sich als dokumentations- und qualitätsorientiertes Projekt an einer bereits funktionierenden Codebasis, ohne dass neue Features entwickelt werden müssen. Der Fokus liegt auf Nachvollziehbarkeit von Architekturentscheidungen und auf Testabdeckung, nicht auf neuer Funktionalität. Das Projekt eignet sich für Studierende, die gerne strukturiert dokumentieren und testen und ein Gespür für nachhaltige Softwarequalität entwickeln möchten.

---

4. Unit Tests im Gymi-Vergleichsportal

Kontext

Das bestehende Vergleichsportal für Gymivorbereitungs-Anbieter basiert auf einem Technologie-Stack aus Next.js, TypeScript, Supabase, Zod und Zustand. Eine Jest-Testinfrastruktur (inkl. ts-jest) ist bereits installiert, jedoch noch keine Tests geschrieben.

Leitfragen

- Welche Teile der Anwendungslogik sind am kritischsten für die Datenqualität und sollten daher primär getestet werden?
- Wie lassen sich die vier Zod-Validierungsschemas systematisch mit Unit Tests abdecken?
- Wie wird die MCDA-Scoring-Logik in adaptProviders.ts sowie die beiden Zustand-Stores zuverlässig getestet, um Fehler in Gewichtung und Datenvalidierung frühzeitig zu erkennen?

Ziel

Aufbau einer systematischen Unit-Test-Abdeckung für die Kernlogik des Portals, mit Fokus auf die vier Zod-Validierungsschemas, die MCDA-Scoring-Logik in adaptProviders.ts sowie die zwei Zustand-Stores.

Ergebnis

Eine vollständige, lauffähige Jest-Testsuite, die die zentrale Validierungs- und Scoring-Logik des Portals absichert und Regressionen frühzeitig sichtbar macht.

Eignet sich als kompaktes, technisch fokussiertes Testing-Projekt, da Jest und ts-jest bereits installiert sind und ohne zusätzlichen Infrastruktur-Aufwand gestartet werden kann. Der Fokus liegt auf dem Schreiben zielgerichteter Unit Tests für geschäftskritische Logik. Das Projekt eignet sich für Studierende, die praktische Erfahrung mit Testautomatisierung in TypeScript/Next.js-Anwendungen sammeln möchten.

---

5. Aktualisierung einer bestehenden Next.js App zu «CS-UBS-Fusion»

Kontext

Basis ist die bestehende Plattform mit Informationen über CS und «CS-UBS-Fusion» Technologie Stack: Next.js, Prisma, MongoDB.

Leitfragen

- Wie lässt sich Next.js auf die aktuelle LTS-Version 16.x mittels Prompting akualsisieren, und wie werden diese sauber aufgefangen?
- Wie wird die im Header bereits vorhandene, aber bislang nicht funktionale Suchfunktion so implementiert, dass sie Fälle, Blogs und Organigramm-Einträge mehrsprachig durchsucht?
- Wie lässt sich eine neue Fallseite «Kapitalcompliance» konsistent zu den bestehenden Archegos-/Greensill-Seiten strukturieren?

Ziel

Aktualisierung der Next.js-Version, Implementierung einer mehrsprachigen Suchfunktion sowie Erstellung einer neuen Fallseite «Kapitalcompliance» auf Basis bereits vorliegender Inhalte.

Ergebnis

Ein lauffähige Web-App mit funktionierender, mehrsprachiger Suche, einer neuen Fallseite sowie ein kurzer Ergebnisbericht zu technischen Entscheidungen, KI-Einsatz und offenen Punkten.

Eignet sich als praxisnahes Erweiterungsprojekt an einer bestehenden Codebasis, bei dem Versions-Upgrade, Feature-Implementierung und Content-Erstellung kombiniert werden. Der Fokus liegt auf sauberer Migration und Feature-Fertigstellung. Das Projekt eignet sich für Studierende, die Erfahrung mit dem Weiterentwickeln und Modernisieren bestehender Next.js-Anwendungen sammeln möchten.

Kurzübersicht

Gruppe	#	Projekt	Schwerpunkt
C	1	LangChain literate programming	LLM-Framework, didaktische Aufbereitung
C	2	LangGraph literate programming	Graphbasierte Multi-Agenten-Architektur
D	1	Tatoeba-API Integration	REST-API-Integration, semantische Disambiguierung
D	2	Vergleich WebScraping-Technologien	Crawl4AI vs. bestehende Scraper, empirischer Vergleich
D	3	Zod/Zustand Dokumentation & Tests	ADRs, Testaufbau, Entwicklerhandbuch
D	4	Unit Tests Gymi-Portal	Testabdeckung für Validierung & Scoring
D	5	Next.js Update «CS-UBS-Fusion»	Versions-Upgrade, Suche, neue Fallseite